# Adaptation-Aware Model Selection for Few-Shot Digital Twin Instantiation

ARTICLE INFO

*Keywords*:
digital twin
few-shot adaptation
model instantiation
architecture selection
model deployment limits
heterogeneous computing centers

ABSTRACT

As an important modeling tool for managing computing centers, digital twin (DT) platforms reuse pre-trained DT models from existing centers to reduce the cost of DT instantiation in large-scale computing networks. Target-center heterogeneity in resource configurations and workload characteristics complicates direct model reuse when target adaptation uses a small support set. However, existing few-shot adaptation methods typically retain a predefined source architecture and therefore do not directly address target-specific model selection for heterogeneous centers under per-model deployment limits. To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an adaptation-aware model-selection algorithm for DT instantiation with few-shot target adaptation under deployment limits. MSA-DTI constructs a reusable candidate bank of source-trained models with diverse architectures, filters candidates that violate the operation-count and parameter-count limits, and adapts the remaining candidates using the same target support set and update budget. It then compares post-adaptation validation performance and replaces the adapted, development-selected reference only when the best-performing alternative achieves a calibrated relative reduction in validation loss. On the Bitbrains workload trace, MSA-DTI reduces aggregate mean squared error (MSE) by 6.88% compared with pre-training and fine-tuning (PT+FT), while satisfying both deployment limits in all evaluated cases.

## 1. Introduction

As an important modeling tool for managing computing centers, a shared digital twin (DT) platform maintains center-specific models for monitoring, prediction, analysis, and decision making [1–4]. In large-scale computing networks, reusing and adapting pre-trained DT models instead of constructing each target model from scratch reduces DT-instantiation cost [5, 6]. Each pre-trained model comprises an architecture and source-trained parameters. The architecture determines the model structure and deployment-related complexity, whereas the parameters provide the initialization for target adaptation. For a new or reconfigured center, differences in resource configurations, monitored-variable availability, workload patterns, and data distributions complicate direct reuse of a source model, while target adaptation uses a small support set. The source architecture is therefore not necessarily suitable for the target center. In addition, large-scale DT platforms impose per-model resource budgets, represented here by limits on estimated operation count per inference and parameter count. Target-center instantiation therefore requires selecting a suitable reusable model and adapting its parameters from the small support set while satisfying both deployment limits. Figure 1 illustrates this setting.

Existing few-shot adaptation methods reuse pre-trained models [7, 8] and have also been applied to DT modeling [6]. However, these methods typically retain the source architecture and therefore do not directly address target-specific model selection for heterogeneous target centers under deployment limits. Deployment-aware architecture-selection methods consider model performance together with deployment cost [9–11], but do not directly address few-shot adaptation of reusable pre-trained models. Methods that jointly consider architecture decisions and few-shot adaptation [12–14] typically rely on target-side architecture or adaptation-structure search. The remaining challenge is therefore to select a suitable reusable pre-trained DT model under per-model deployment limits and adapt its parameters from a small target support set.

To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an adaptation-aware model-selection algorithm for DT instantiation with few-shot target adaptation under deployment limits. MSA-DTI first constructs a reusable candidate bank of source-trained models with diverse architectures. For a target center, it filters candidates that violate the deployment limits and adapts the remaining feasible candidates under a common adaptation budget. It then compares the adapted candidates using validation performance and retains a development-selected reference model unless the best-performing alternative achieves a sufficient reduction in validation

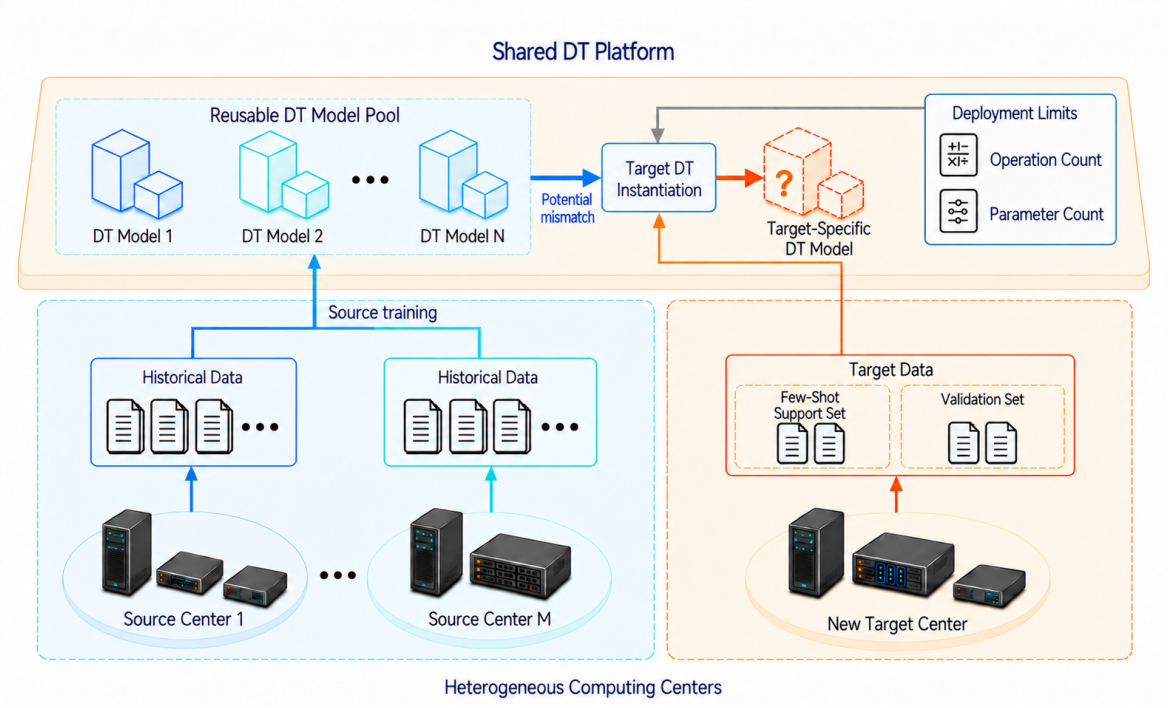



Figure 1: System setting for target-specific DT instantiation.

loss. Thus, model selection is based on post-adaptation performance without target-side search for new architectures or adaptation structures.

The main contributions are summarized below.

- We formulate DT instantiation as jointly selecting among reusable source-trained models with different architectures and adapting the selected model's parameters from a small target support set, subject to hard per-model operation-count and parameter-count limits.
- We propose a feasibility-first, adaptation-aware model-selection strategy that filters out infeasible candidates before target adaptation and compares the remaining candidates only after adapting them using the same target support set and update budget.
- We introduce a reference-model replacement rule with a relative-improvement threshold calibrated separately from reference designation. An adapted alternative replaces the development-selected reference only when it achieves the calibrated relative reduction in validation loss.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the system model and problem. Section 4 presents MSA-DTI. Section 5 reports the experiments, and Section 6 concludes the paper.

## 2. Related Work

### 2.1. Digital Twin Modeling and Platform Deployment

In communication and computing systems, digital twins represent system states and services while supporting model updates [5, 15–17]. At the deployment level, related work considers service orchestration and computation offloading under resource constraints [18, 19]. Together, these strands motivate jointly considering model suitability and deployment feasibility in target-specific DT instantiation.

### 2.2. Limited-Data Model Adaptation

Limited-data methods leverage source or prior knowledge for target-model adaptation [7, 8, 20]. Recent approaches improve cross-task and cross-domain adaptation through context modeling and transferable representations [21–24].

Limited-data learning has also been applied to DT construction and workload prediction [6, 25]. Most parameter-adaptation methods operate with a fixed architecture. For heterogeneous target centers, a fixed architecture is not necessarily suitable, motivating selection among reusable source-trained models.

Pre-trained model selection has also been studied in model hubs: LogME ranks candidates by estimated transferability before fine-tuning [26], while B-Tuning uses multiple highly ranked pre-trained models to guide target-model tuning [27].

Model selection introduces a related challenge because candidate rankings based on validation estimates are subject to uncertainty [28, 29]. This motivates a conservative decision rule when adapted reusable models are compared on target validation data.

### 2.3. Deployment-Aware Architecture Selection

Deployment-aware neural architecture search (NAS) evaluates model performance together with deployment cost [9–11]. Few-shot, zero-shot, and training-free NAS variants reduce the cost of architecture evaluation during search [30–32]. In Ref. [30], "few-shot" denotes evaluation with several sub-supernets rather than adaptation from a small target support set.

Among meta-learning-based NAS methods, T-NAS learns transferable architecture knowledge for new-task adaptation [13]. H-Meta-NAS further incorporates hardware constraints into task-adaptive search [12]. Neural fine-tuning search instead starts from a pre-trained backbone and searches over adaptation structures, including which components to fine-tune or augment [14]. Together, these methods extend task adaptation beyond parameter updates to architecture or adaptation-structure decisions.

To address target-specific model selection under deployment limits without target-stage search for new architectures or adaptation structures, MSA-DTI selects among a finite bank of separately source-trained reusable models using deployment feasibility and post-adaptation performance.

## 3. System Model and Problem Formulation

### 3.1. System Setting and Target Model

We consider a shared DT platform that maintains center-specific models for heterogeneous computing centers.

Let $c$ denote a new or reconfigured target center. To enable model reuse across centers with different monitored-variable availability, all observations follow a common metric schema, with unavailable variables represented by masks rather than omitted. Let $\mathcal{X}$ denote the common input space defined by this metric schema, and let $\mathcal{Y}$ denote the output space of the DT task. Let $\mathcal{A}$ denote the set of model architectures available to the shared DT platform. For $a \in \mathcal{A}$, let $\Theta(a)$ denote the trainable-parameter space associated with architecture $a$, and let $\theta \in \Theta(a)$ denote a particular parameter setting. A model is represented as

$$f_{a,\theta} : \mathcal{X} \to \mathcal{Y}, \qquad a \in \mathcal{A},\ \theta \in \Theta(a). \tag{1}$$

The DT model instantiated for center $c$ is represented as

$$\mathrm{DT}_c = \left(a_c, \theta_c\right), \qquad a_c \in \mathcal{A},\ \theta_c \in \Theta(a_c), \tag{2}$$

where $a_c$ and $\theta_c$ denote the target architecture and target-specific parameters, respectively. During instantiation, the available target data are represented by a support set and a validation set.

$$\begin{aligned} \mathcal{D}_c^{\mathrm{sup}} &= \left\{ (x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}}) \right\}_{i=1}^{K_c}, \\ \mathcal{D}_c^{\mathrm{val}} &= \left\{ (x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}}) \right\}_{j=1}^{V_c}, \\ \mathcal{D}_c^{\mathrm{tar}} &= \mathcal{D}_c^{\mathrm{sup}} \cup \mathcal{D}_c^{\mathrm{val}}, \qquad \mathcal{D}_c^{\mathrm{sup}} \cap \mathcal{D}_c^{\mathrm{val}} = \varnothing. \end{aligned} \tag{3}$$

In these sets, $(x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}})$ and $(x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}})$ denote the $i$th support and $j$th validation input-output pairs, respectively. Their union $\mathcal{D}_c^{\mathrm{tar}}$ denotes the target data available during instantiation. Here, $K_c, V_c \in \mathbb{N}_+$ denote the support and validation sample counts, respectively. Support samples are used for few-shot parameter adaptation, whereas validation samples are reserved for post-adaptation candidate comparison. The validation set available at a decision time contains

only windows whose complete prediction horizon has already been observed. Future evaluation labels are excluded from $\mathcal{D}_c^{\mathrm{tar}}$. The present study concerns determining $(a_c, \theta_c)$ during target-center instantiation. Subsequent model updates are outside its scope.

### 3.2. Reusable Models and Deployment Limits

The shared DT platform maintains source-trained models learned from observations of existing centers. Each reusable model consists of a model architecture and its source-trained parameter initialization and serves as a candidate for target-center instantiation. Let the finite set of $N$ reusable source-trained models be

$$\mathcal{Q} = \{q_n\}_{n=1}^{N}, \qquad q_n = (a_n, \theta_n^0), \tag{4}$$

where $q_n$ denotes the $n$th reusable source-trained model, $a_n \in \mathcal{A}$ denotes its architecture, and $\theta_n^0 \in \Theta(a_n)$ denotes its source-trained initialization. The architectures represented in $\mathcal{Q}$ form the set $\mathcal{A}_{\mathcal{Q}} = \{a_n\}_{n=1}^{N} \subseteq \mathcal{A}$.

For target center $c$, the platform specifies two per-model deployment limits. They are a maximum estimated operation count for one batch-size-one forward pass and a maximum number of trainable parameters. Let $B_c^F$ and $B_c^P$ denote these limits, respectively. For $a \in \mathcal{A}_{\mathcal{Q}}$, let $F_c(a)$ denote the estimated operation count under the fixed counting rule, and let $P_c(a)$ denote the exact number of trainable parameters. In this formulation, parameter adaptation changes only parameter values while keeping the architecture fixed. Therefore, $F_c(a)$ and $P_c(a)$ remain unchanged.

For center $c$, the deployment-feasible reusable model set is

$$\mathcal{Q}_c^{\mathrm{fea}} = \left\{ q_n \in \mathcal{Q} \;\middle|\; F_c(a_n) \le B_c^F,\ P_c(a_n) \le B_c^P \right\}. \tag{5}$$

Both deployment limits are modeled as hard constraints rather than penalty terms.

### 3.3. DT Model Instantiation Problem

Let $\mathbb{P}_c$ denote the target-center data distribution over $\mathcal{X} \times \mathcal{Y}$, and let $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\ge 0}$ denote the task loss function. The target risk is defined as

$$R_c(a, \theta) = \mathbb{E}_{(x,y)\sim\mathbb{P}_c}\left[\ell(f_{a,\theta}(x), y)\right]. \tag{6}$$

Let $\mathcal{U}_c$ denote a prescribed target-adaptation operator. For each reusable model $q_n = (a_n, \theta_n^0)$, $\mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}}) \in \Theta(a_n)$ denotes the target-specific parameters obtained by adapting $q_n$ using the target support set. When $\mathcal{Q}_c^{\mathrm{fea}} \neq \emptyset$, an ideal reusable-model decision for center $c$ is defined as

$$\begin{aligned} n_c^{\dagger} \in \underset{n\in\{1,\dots,N\}}{\arg\min} \quad & R_c\left(a_n, \mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}})\right) \\ \text{s.t.} \quad & F_c(a_n) \le B_c^F, \\ & P_c(a_n) \le B_c^P. \end{aligned} \tag{7}$$

Here, $n_c^{\dagger}$ denotes the reusable-model index selected under the constrained target-risk criterion in Eq. (7). The corresponding DT model is $\left(a_c^{\dagger}, \theta_c^{\dagger}\right) = \left(a_{n_c^{\dagger}}, \mathcal{U}_c(q_{n_c^{\dagger}}, \mathcal{D}_c^{\mathrm{sup}})\right)$. The target distribution $\mathbb{P}_c$ is unavailable during instantiation. The target risk $R_c$ therefore cannot be evaluated directly. The decision must instead be based on the support and validation data in Eq. (3).

## 4. The Proposed MSA-DTI Algorithm

### 4.1. Overview

Figure 2 summarizes MSA-DTI for the instantiation problem in Eq. (7). The method contains an offline preparation stage and a target-side instantiation stage. Offline preparation constructs the reusable candidate bank $\mathcal{Q}$, selects the reference candidate $q^{\mathrm{ref}}$ from development outcomes, and calibrates the relative margin $\tau$ on a separate calibration pool. Both pools are disjoint from final target evaluation. For a target center, the target-side stage filters candidates by the two deployment limits, adapts the feasible candidates using the same support set and update budget, and compares their post-adaptation performance on the validation set. The best alternative replaces the adapted reference only when it satisfies the calibrated relative-improvement requirement. The output is the target-specific DT model $\mathrm{DT}_c$.

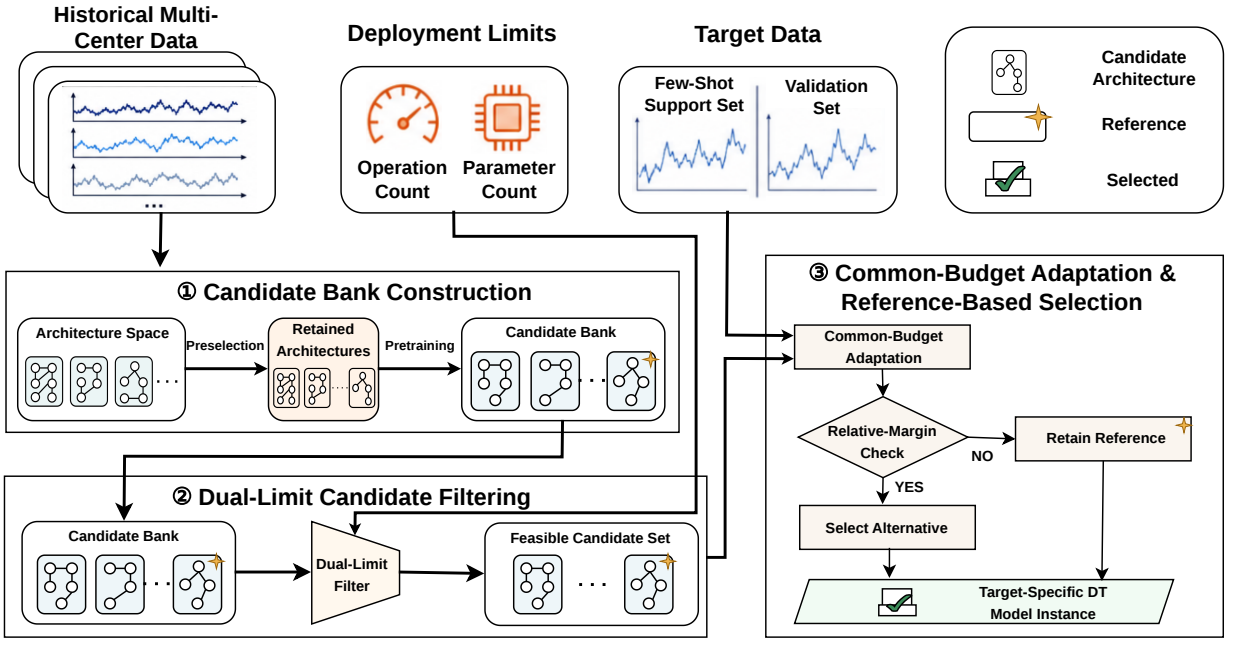



Figure 2: Overall workflow of MSA-DTI. Architecture icons are schematic.

## 4.2. Candidate Bank Construction

Before constructing the final source-trained candidate bank, MSA-DTI performs architecture preselection on a dedicated pool of architecture-development cases, separate from the data used for reference selection, margin calibration, and final target evaluation. A preselection reference architecture $a^{\mathrm{pre}}$ is fixed and retained. Each case contains disjoint support, validation, and check subsets. The preselection reference must satisfy the case's deployment limits, and only feasible candidates enter the comparison. The preselection reference and feasible candidates are adapted from fixed preliminary source-trained initializations using support data; validation and check losses are then evaluated.

Two evidence counts are accumulated for each preselection candidate architecture. A *check-oracle win* is assigned to the feasible candidate with the lowest check loss when that loss is lower than the adapted preselection reference's check loss. A *validation-selected positive check win* is assigned when a candidate is selected from the union of the preselection reference and feasible candidates by validation loss, and its check loss is lower than that of the preselection reference. A candidate architecture is retained if either count reaches a preset threshold.

Let $\mathcal{D}_{\mathrm{src}}$ denote the combined source data. For each retained architecture $a_q$, source training produces an initialization $\theta_q^0$, forming a record $q = (a_q, \theta_q^0) \in \mathcal{Q}$. Distinct source-trained initializations of the same architecture form distinct candidates. When source checkpoints are task-specific, a fixed candidate identifier resolves to the checkpoint associated with the corresponding task. Figure 3 summarizes this construction and the subsequent reference selection. The depicted architectures and indices are illustrative; shading distinguishes source initializations for the same architecture. The highlighted reference identifies a candidate selected from the completed bank.

Reference selection is a separate offline decision over this completed bank. Let $\mathcal{Q}_{\mathrm{all}}$ contain the candidate identifiers whose task-specific models satisfy both deployment limits for every prespecified task configuration and deployment budget. Let $\mathcal{V}_{\mathrm{dev}}$ denote the nonempty set of development target instances and $\mathcal{H}_v$ the nonempty set of prespecified tasks associated with instance $v$. For candidate $q$, $Y_{v,h}^{\mathrm{real}}(q)$ denotes its post-adaptation loss measured on the designated development assessment data for task $h$. Its average development loss is

$$\overline{Y}_{\mathrm{dev}}(q) = \frac{1}{|\mathcal{V}_{\mathrm{dev}}|} \sum_{v \in \mathcal{V}_{\mathrm{dev}}} \frac{1}{|\mathcal{H}_v|} \sum_{h \in \mathcal{H}_v} Y_{v,h}^{\mathrm{real}}(q), \qquad q \in \mathcal{Q}_{\mathrm{all}}. \tag{8}$$

Reference designation requires $\mathcal{Q}_{\mathrm{all}} \neq \emptyset$ and uses

$$q^{\mathrm{ref}} \in \arg\min_{q \in \mathcal{Q}_{\mathrm{all}}} \overline{Y}_{\mathrm{dev}}(q). \tag{9}$$

Exact ties are resolved by maximum task-specific parameter count, maximum task-specific operation count, and candidate token, in ascending order. Let $r$ index the selected reference record, written as $q^{\mathrm{ref}} = (a^{\mathrm{ref}}, \theta_r^0)$ for the corresponding task. Its identifier is fixed before calibration and evaluation and shared across target instances. Its parameters are adapted to each target's support observations, just as for every other feasible candidate. This stage outputs $\mathcal{Q}$ and $q^{\mathrm{ref}}$ for target-side instantiation.

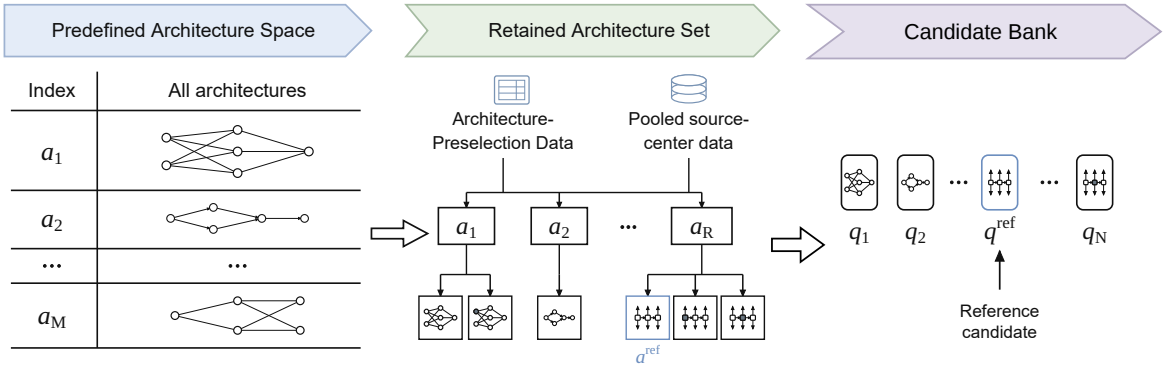



Figure 3: Candidate-bank construction and reference designation. In this schematic, $M$ and $R$ denote the numbers of predefined and retained architectures, respectively, and $N$ denotes the number of candidate records; the retained architecture indices are illustratively renumbered after preselection.

### 4.3. Target-Side Instantiation

For target center $c$, MSA-DTI computes $\mathcal{Q}_c^{\mathrm{fea}}$ using Eq. (5) and restricts target adaptation to the candidates in this set. Because the subsequent reference-based selection requires a deployment-feasible reference for comparison and fallback, MSA-DTI proceeds only if $q^{\mathrm{ref}} \in \mathcal{Q}_c^{\mathrm{fea}}$. Otherwise, it terminates without DT instantiation under the current deployment limits.

For target-center instantiation, the support set $\mathcal{D}_c^{\mathrm{sup}}$ is used for parameter adaptation, whereas the validation set $\mathcal{D}_c^{\mathrm{val}}$ is used for post-adaptation candidate comparison. For a dataset $\mathcal{D}$, let

$$\mathcal{L}(\mathcal{D}; a, \theta) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \ell\left(f_{a,\theta}(x), y\right) \tag{10}$$

denote the empirical task loss.

For each feasible candidate $q = (a_q, \theta_q^0)$, let $\theta_{c,q}^{(t)}$ denote its parameters after $t$ target-side adaptation steps. With a common update budget of $T$ steps and a learning rate $\eta$ shared by all feasible candidates, the adaptation is

$$\begin{aligned} \theta_{c,q}^{(0)} &= \theta_q^0, \\ \theta_{c,q}^{(t+1)} &= \theta_{c,q}^{(t)} - \eta \nabla_\theta \mathcal{L}\left(\mathcal{D}_c^{\mathrm{sup}}; a_q, \theta_{c,q}^{(t)}\right), \\ & \qquad t = 0, \ldots, T-1. \end{aligned} \tag{11}$$

Equation (11) gives the gradient-based realization of the target-adaptation operator $\mathcal{U}_c$ introduced in Section 3.3, with $\theta_{c,q}^{(T)}$ as its output. The implementation clips the gradient norm as specified in the experimental setup. After $T$ adaptation steps, let $J_c(q)$ denote the post-adaptation validation score of candidate $q$.

$$J_c(q) = \mathcal{L}\left(\mathcal{D}_c^{\mathrm{val}}; a_q, \theta_{c,q}^{(T)}\right). \tag{12}$$

Because validation-based candidate rankings are subject to estimation uncertainty, MSA-DTI uses a calibrated relative-improvement margin to govern replacement of the reference candidate.

Let $\tau \in [0, 1)$ be the calibrated relative margin. The admissible set is

$$\mathcal{A}_c(\tau) = \{q^{\mathrm{ref}}\} \cup \{q \in \mathcal{Q}_c^{\mathrm{fea}} \setminus \{q^{\mathrm{ref}}\} : J_c(q) \le (1-\tau) J_c(q^{\mathrm{ref}})\}. \tag{13}$$

The selected candidate $q_c^*$ minimizes $J_c(q)$ over this set, resolving exact ties by parameter count, operation count, architecture index, and candidate identifier, all in ascending order. If the calibrated policy is reference-only, $q_c^* = q^{\mathrm{ref}}$.

Selection requires finite, strictly positive validation scores for all feasible candidates; values outside this domain trigger a numerical-domain failure.

A larger $\tau$ requires a larger validation-loss reduction for an alternative to be selected over the reference. The margin affects only model selection and does not change the adapted parameters.

The selected candidate $q_c^*$ provides the architecture $a_{q_c^*}$, and its adapted parameters $\theta_{c,q_c^*}^{(T)}$ define the target-specific DT model in Eq. (2). This stage outputs $\mathrm{DT}_c$. Figure 4 illustrates common-budget adaptation and the reference-based selection rule. Candidate labels identify the same models before and after parameter adaptation; the candidate identities and loss values shown are illustrative.

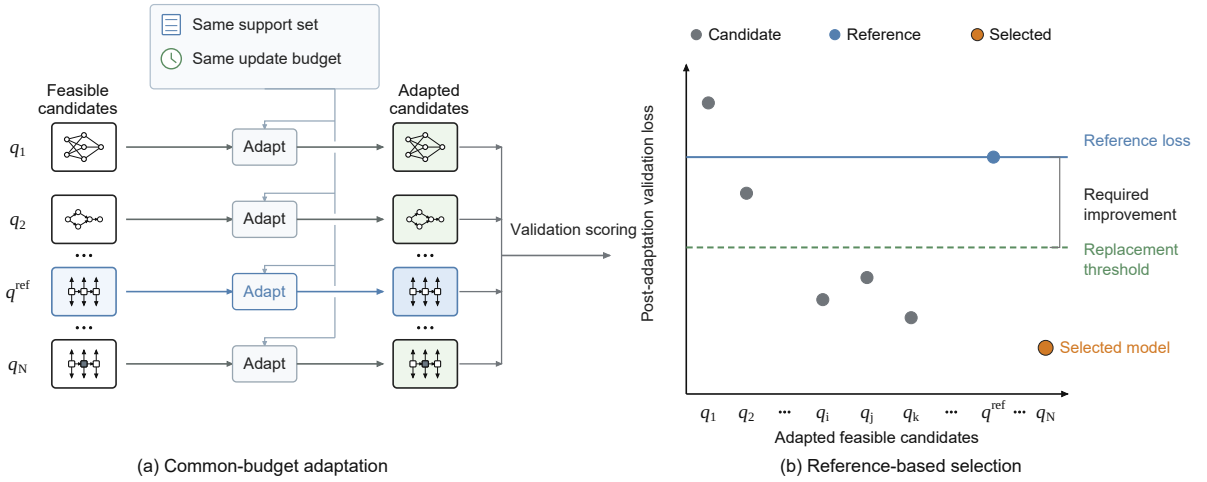



Figure 4: Common-budget adaptation and reference-based selection.

Algorithm 1 summarizes the target-side procedure.

**Algorithm 1** Target-Side MSA-DTI for One Center

**Require:** Candidate bank $\mathcal{Q}$, reference candidate $q^{\mathrm{ref}}$, support set $\mathcal{D}_c^{\mathrm{sup}}$, validation set $\mathcal{D}_c^{\mathrm{val}}$, deployment limits $B_c^F$ and $B_c^P$, update budget $T$, learning rate $\eta$, frozen policy, and calibrated relative margin $\tau$

**Ensure:** Target-specific DT model $\mathrm{DT}_c$, or termination without instantiation

Compute $\mathcal{Q}_c^{\mathrm{fea}}$ using Eq. (5)
**if** $q^{\mathrm{ref}} \notin \mathcal{Q}_c^{\mathrm{fea}}$ **then**
    terminate without DT instantiation
**end if**
**for all** $q \in \mathcal{Q}_c^{\mathrm{fea}}$ **do**
    Adapt $q$ using Eq. (11)
    Compute $J_c(q)$ using Eq. (12)
    **if** $J_c(q)$ is nonfinite or $J_c(q) \le 0$ **then**
        terminate with a numerical-domain failure
    **end if**
**end for**
**if** the frozen policy is reference-only **then**
    $q_c^* \leftarrow q^{\mathrm{ref}}$
**else**
    Form $\mathcal{A}_c(\tau)$ using Eq. (13)
    Select $q_c^* \in \arg\min_{q \in \mathcal{A}_c(\tau)} J_c(q)$, using the deterministic tie-breaking order specified above
**end if**
$\mathrm{DT}_c \leftarrow \left(a_{q_c^*}, \theta_{c,q_c^*}^{(T)}\right)$
**return** $\mathrm{DT}_c$

### 4.4. Relative-Margin Calibration and Analytical Properties

The margin $\tau$ is calibrated on a pool disjoint from the data used for architecture preselection, reference designation, and final evaluation. Each calibration case uses support data for adaptation and mature validation data for model selection. A chronologically later outcome segment provides the loss used to evaluate the selected model during offline calibration.

Let $\mathcal{T} \subset [0, 1)$ be a preset finite margin set. Let $\widehat{R}_{\mathrm{harm}}(\tau)$ denote the empirical harmful-selection rate and $\widehat{L}(\tau)$ the mean outcome loss of the selected models. Both quantities are averaged first over tasks within each calibration target instance and then equally across instances. A case contributes one to the harm indicator when the selected alternative has a higher outcome loss than the adapted reference, and zero otherwise. Let $\widehat{L}_{\mathrm{ref}}$ denote the corresponding mean outcome loss when the adapted reference is retained in every case. For a preset empirical harm limit $\rho$, the eligible set is

$$\mathcal{T}_{\mathrm{elig}} = \{\tau \in \mathcal{T} : \widehat{R}_{\mathrm{harm}}(\tau) \le \rho\}. \tag{14}$$

Among eligible margins, select a minimizer of $\widehat{L}(\tau)$, breaking exact ties toward the larger margin. If $\mathcal{T}_{\mathrm{elig}}$ is empty or its minimum loss is not strictly lower than $\widehat{L}_{\mathrm{ref}}$, the policy is reference-only. Otherwise the selected margin is fixed before evaluation. The harm constraint is evaluated empirically on the calibration pool.

By construction, any DT model instantiated by MSA-DTI is selected from $\mathcal{Q}_c^{\mathrm{fea}}$ and therefore satisfies both deployment limits. Moreover, whenever an alternative is selected and $J_c(q^{\mathrm{ref}}) > 0$, Eq. (13) guarantees a relative validation-loss reduction of at least $\tau$ with respect to the reference.

With resource counts precomputed for the relevant task configurations, the target-stage computational complexity is

$$O\left(|\mathcal{Q}| + \sum_{q \in \mathcal{Q}_c^{\mathrm{fea}}} \left(T C_q^{\mathrm{grad}} + C_q^{\mathrm{val}}\right)\right). \tag{15}$$

Here, $C_q^{\mathrm{grad}}$ and $C_q^{\mathrm{val}}$ denote the costs of one target update and validation evaluation for candidate $q$, respectively. The $|\mathcal{Q}|$ term accounts for deployment filtering.

## 5. Experimental Evaluation

The experiments assess prediction accuracy, selection reliability, deployment feasibility, and target-side computation. The primary comparison uses the medium deployment budget; tight and loose budgets examine resource sensitivity.

### 5.1. Experimental Setup

*Workload and prediction tasks.* The experiments use the Bitbrains fastStorage workload trace from the Grid Workloads Archive [33, 34]. Source training, reference selection, margin calibration, and evaluation use separate groups of 20, 20, 20, and 30 VMs, respectively. Each VM defines a target workload prediction task, providing a trace-based instance of the DT workload-model setting. Windowing and causal feature construction map the recorded measurements to this setting. The analysis is a retrospective, non-blind comparison on a single workload trace, with uncertainty assessed across VMs.

Each input contains 96 consecutive records and predicts CPU utilization over $H \in \{1, 4\}$ future records. Parameter adaptation uses $K \in \{10, 20\}$ labeled support windows. A subsequent validation history provides 1342–1351 complete windows before validity masking, all with fully observed prediction horizons at the selection time. The four $H/K$ settings produce 80 calibration tasks and 120 evaluation tasks. Each selected model then makes 621–627 rolling-origin forecasts with fixed parameters. Newly arriving observations enter forecast inputs; future evaluation labels do not enter adaptation or model selection. Thus, the few-shot budget refers to the support labels used for parameter updates; model selection also uses the separate validation history.

The target is recorded CPU utilization divided by 100. Inputs comprise ten monitored metrics, two causal CPU features, twelve validity indicators, and a timestamp-gap channel. Invalid measurements are represented by a validity mask and a forward-filled value, initialized at zero. Input metrics other than CPU utilization percentage use source-fitted log scaling. Every method is scored on the same windows with complete, valid target horizons. Table 1 summarizes the main experimental settings.

*Candidate bank and calibration.* Architecture preselection evaluates 66 configurations on 80 controlled synthetic development cases and retains six declared configurations using the evidence-count threshold of two. The subsequent

Table 1: Experimental configuration.

| Item | Setting |
|---|---|
| VM groups | Source / development / calibration / evaluation: 20 / 20 / 20 / 30 |
| Target input and output | 96 records, 25 channels; CPU utilization over $H \in \{1, 4\}$ records |
| Adaptation support | $K \in \{10, 20\}$ windows; 50 SGD updates, learning rate 0.01 |
| Candidate bank | Seven source-trained records; five executable structures |
| Reference and margin | Development-selected A55; calibration-selected $\tau = 0.05$ |
| Budgets | Tight / medium / loose: operations $1.5/5/20 \times 10^6$; parameters $3/10/50 \times 10^4$ |
| Primary evaluation | Medium budget; 120 tasks clustered within 30 VMs |

source training uses Bitbrains observations. Independent source training produces seven candidate records: A1, A6, A13, A55, A56, A57-C1, and A57-PT. They comprise three MLP depths and two one-layer GRU sizes. A56, A57-C1, and A57-PT share the executable GRU-32 structure but have distinct source initializations. Each record has a checkpoint for each horizon. A55 (GRU-16) minimizes the development loss among candidates feasible under all task configurations and budgets and is designated as the reference.

All bank candidates use the same full-support SGD adaptation: 50 updates, MSE loss, learning rate 0.01, and gradient-norm clipping at 1. Calibration uses $\mathcal{T} = \{0, 0.05, 0.075, 0.10, 0.125, 0.15, 0.20\}$ and empirical harm limit $\rho = 0.05$ at the medium budget, yielding $\tau = 0.05$. The calibrated policy is applied at all three budgets. The operation-count limits are $1.5 \times 10^6$, $5 \times 10^6$, and $2 \times 10^7$; the corresponding parameter-count limits are $3 \times 10^4$, $10^5$, and $5 \times 10^5$. These limits specify per-model inference complexity.

*Comparison methods.* Seven task-matched baselines cover fixed-model adaptation and target-side architecture selection (Table 2). MeDeT-style adaptation uses source meta-training motivated by Ref. [6]; the H-Meta-NAS-based baseline follows hardware-constrained task-adaptive search [12]. Meta+NAS-lite combines source and support-proxy rankings. The zero-shot baselines use a support-entropy ranking proxy. These names identify the implemented workload-prediction recipes summarized in the table. All methods share the task inputs, available target labels, resource-counting convention, and deployment budgets. Each method retains its specified initialization, optimizer, and candidate-evaluation procedure.

Table 2: Task-matched baseline implementations.

| Method | Initialization and selection | Target updates |
|---|---|---|
| PT+FT | A57-PT source-trained initialization | SGD/MSE, 50 |
| MeDeT-style | A57 meta-initialization; 400 source outer iterations | SGD/MSE, 50 |
| Scratch50 | Randomly initialized A57 | SGD/MSE, 50 |
| Meta+NAS-lite | Up to 12 candidates from 66 configurations; rank $0.6\, r_{\text{source}} + 0.4\, r_{\text{support}}$ | Adam/Huber, 50 per candidate |
| Zero-shot NAS | Top support-entropy proxy; source-trained C1 initialization | 0 |
| Zero-shot NAS+FT | Up to 12 support-proxy-ranked C1 candidates | Adam/Huber, 50 per candidate |
| H-Meta-NAS-based | 240 source outer iterations; two populations of at most six feasible candidates | SGD/MSE, 50 per evaluation |

SGD: learning rate 0.01, gradient norm limit 1. Adam: learning rate $3 \times 10^{-4}$, Huber threshold 0.2, gradient norm limit 5. Adapted search candidates are ranked by validation MSE, then resource counts and architecture index.

*Metrics and uncertainty.* MSE and MAE average prediction errors over valid forecast windows and horizon steps. Within each task, Worst-10% averages the largest $\max\{1, \text{round}(0.1n)\}$ window MSEs, where $n$ is the valid-window count. These metrics are averaged over the four tasks of each VM and then equally across VMs. Task-level CVaR90 averages the largest 12 MSEs among the 120 evaluation tasks. It measures variation across tasks, whereas Worst-10% measures the tail within a task.

The aggregate MSE reduction is $100(1 - \overline{E}_{\text{MSA}}/\overline{E}_{\text{PT+FT}})$. The mean task-relative reduction instead averages $100(1 - E_{\text{MSA},i}/E_{\text{PT+FT},i})$ over paired tasks. Descriptive 95% percentile intervals use 10000 paired VM bootstrap draws (PCG64 seed 20261026), keeping all four tasks of each sampled VM together. Beneficial and harmful selections

compare a selected alternative with the adapted reference on future-window MSE; retaining the reference contributes zero to both counts. The denominator is all evaluation tasks.

## 5.2. Overall Performance and Selection Reliability

Figure 5 reports the medium-budget results. MSA-DTI has the smallest point estimate for all four error measures among the seven baselines. Its aggregate MSE is 0.004488, compared with 0.004820 for PT+FT, a reduction of 6.88% (95% paired VM bootstrap interval: 4.26–15.95%). The corresponding reductions in MAE, Worst-10%, and task-level CVaR90 are 13.29%, 6.55%, and 4.07%, respectively.

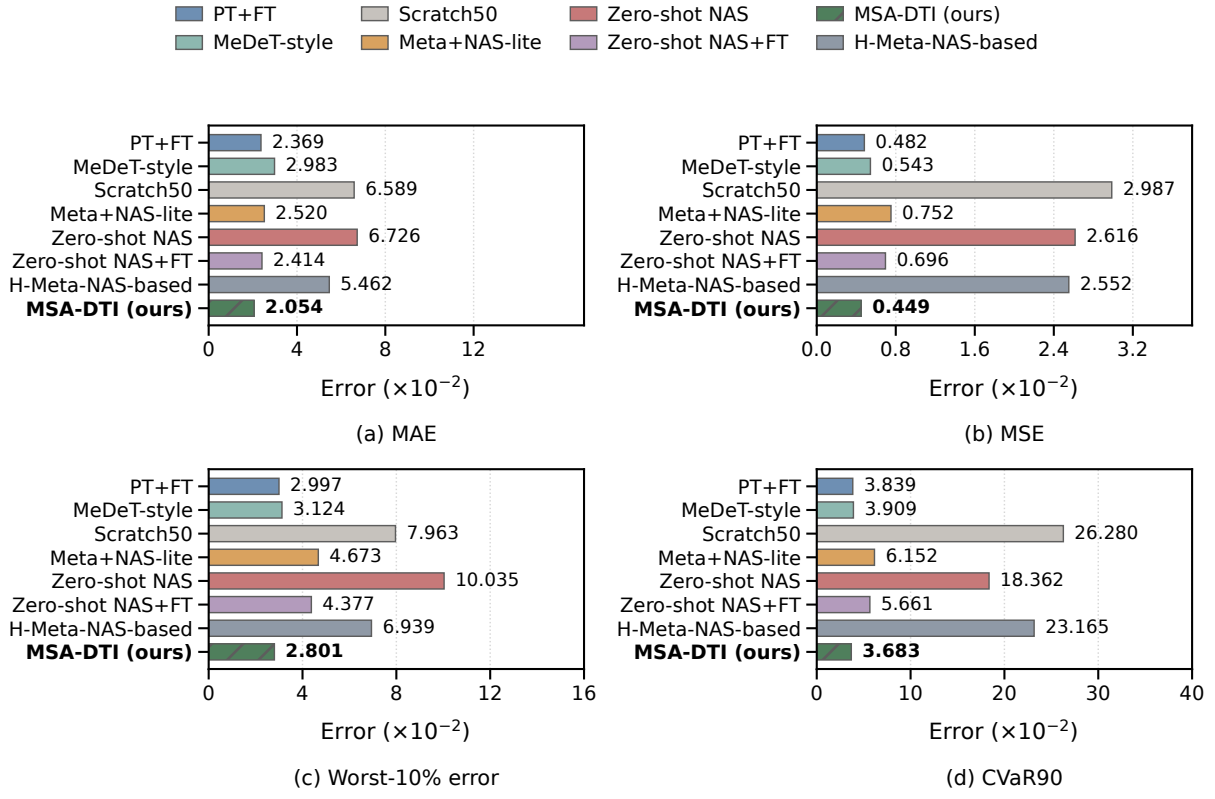



Figure 5: Prediction errors at the medium deployment budget.

The mean task-relative MSE reduction is 41.26%. This statistic gives equal weight to each task's relative change, whereas the 6.88% aggregate reduction reflects the ratio of mean errors. At the VM level, MSA-DTI improves mean MSE over PT+FT for 28 VMs and increases it for two. Figure 6 displays all paired task outcomes. Its comparison axes use PT+FT, while its selection categories use the adapted A55 reference: 84 tasks select a beneficial alternative, five select a harmful alternative, and 31 retain A55. This gives a harmful-selection rate of 4.17% across the 120 tasks.

## 5.3. Robustness and Sensitivity Analysis

Figure 7 groups the paired task-relative MSE reductions by horizon, support size, and deployment budget. The policy is calibrated once at the medium budget. Medium and loose budgets retain the same seven candidates and therefore produce identical MSA-DTI predictions. The tight budget retains four candidates and yields aggregate MSE 0.004532. All subgroup intervals resample VMs, preserving dependence among their tasks.

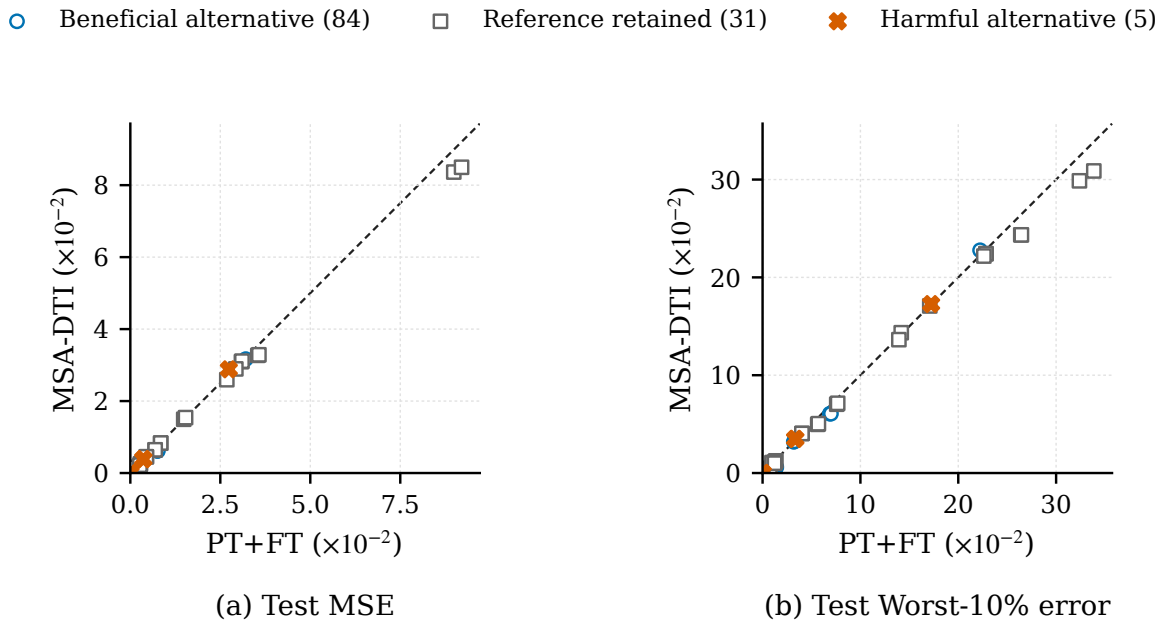



Figure 6: Paired task errors relative to PT+FT: (a) MSE and (b) Worst-10% error. Marker categories are defined by MSE relative to the adapted reference.

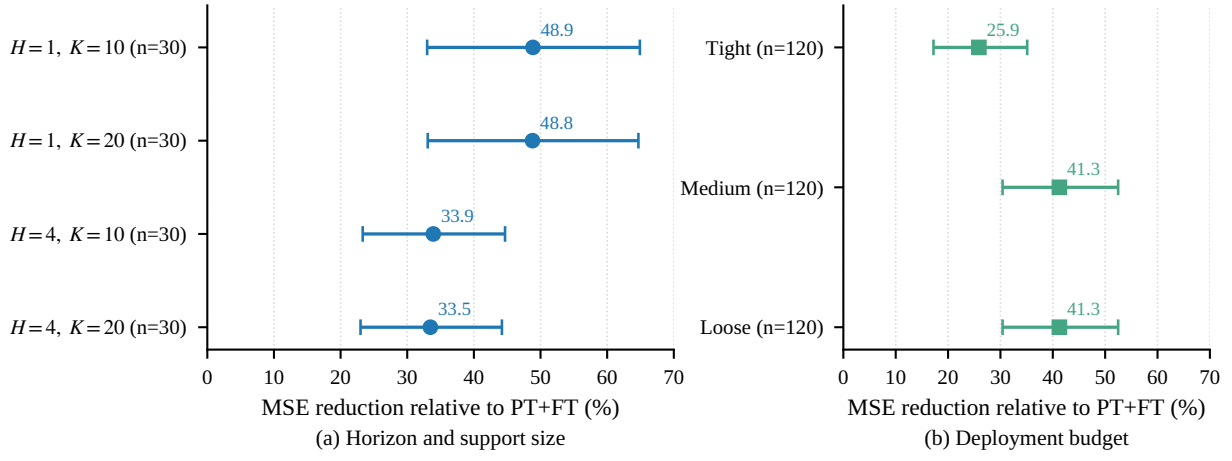
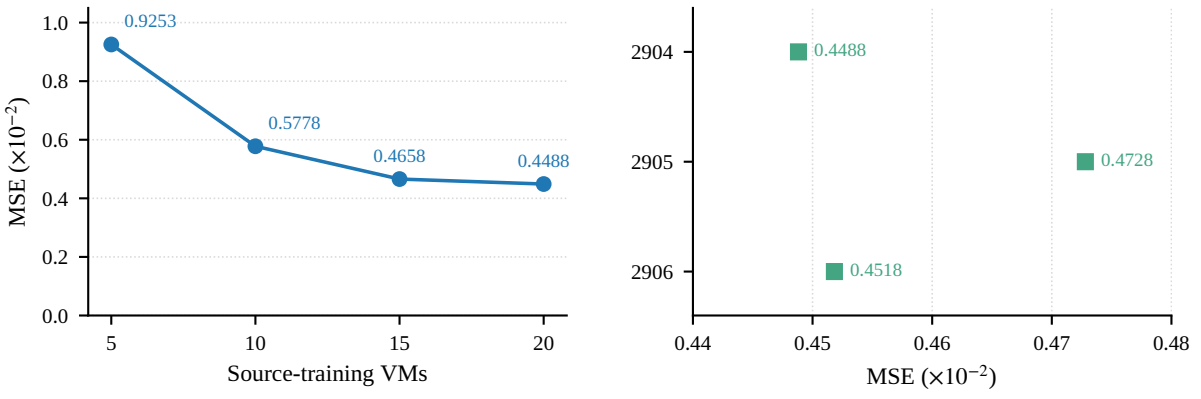



Figure 7: Mean task-relative MSE reduction against PT+FT. Error bars show 95% paired VM-bootstrap intervals.

Source sensitivity uses nested training pools of 5, 10, 15, and 20 VMs at seed 2904 and three training seeds for the 20-VM pool (Fig. 8). Every bank receives its own development-selected reference and calibration-selected margin; feature normalization remains fitted to the 20-VM source pool. Increasing the training pool from five to 20 VMs reduces aggregate MSE from 0.009253 to 0.004488. Across seeds 2904, 2905, and 2906, MSE is 0.004488, 0.004728, and 0.004518, respectively. Their harmful-selection rates are 4.17%, 11.67%, and 5.83%. The latter two rates exceed the 5% calibration criterion, showing that the criterion's empirical calibration bound does not transfer as a future-risk guarantee.

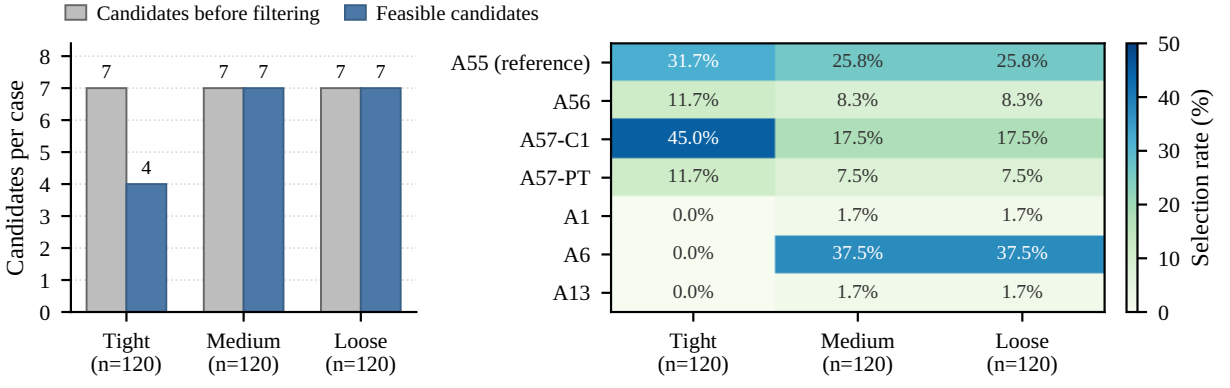



(a) Source-pool size (b) Source-training seed

Figure 8: Source-training sensitivity to (a) source-pool size and (b) training seed. The panels use different MSE scales.

## 5.4. Deployment Feasibility and Cost

All selected models satisfy both deployment limits in every evaluated task and budget. Figure 9 shows the feasible candidate counts and the selection frequencies of individual source-trained records. Under the tight budget, the parameter limit removes the three MLPs; the operation limit alone retains all seven records. With temporal-convolution candidate A24 added, an eight-candidate stress test uses limits of 1,124,496 operations and 79,554.5 parameters. When calibrated separately, operation-only and parameter-only selection violate the omitted limit in two and 16 tasks, respectively. The two constraints therefore control distinct aspects of deployable model complexity.




(a) Candidate filtering (b) Final model selection

Figure 9: Candidate feasibility and model selection under different deployment budgets.

Table 3 reports selected-model complexity and measured target-pipeline cost. The timed implementation adapts all seven bank candidates and then applies each budget's feasibility mask. Its 8.124 s mean therefore measures this complete seven-candidate execution. Equation (15) describes the feasibility-first algorithm, whose adaptation work depends on the feasible subset.

The implementation uses Python and PyTorch on an NVIDIA RTX 3060 Laptop GPU. Timing uses Cal VM 1204, all four $H/K$ settings, and three budgets, with five measured repetitions after one warmup per setting. The synchronized timer covers checkpoint loading, model construction, search proxies, adaptation, validation, filtering, and selection. Prefix preparation, offline training, and future inference are separate stages. The values characterize this workload on the measured host. Separately measured single-window forward latencies for A55 at $H = 1$ are 1.938 ms on CPU and 5.956 ms on GPU. Device and execution overhead therefore matter alongside operation count when translating a deployable model into a running DT service.

Table 3: Model complexity and measured target-pipeline cost.

| Method | Params. ($10^3$) | Ops. ($10^6$) | Adapt. | Mean (s) | Range (s) |
|---|---|---|---|---|---|
| PT+FT | 5.75 | 1.051 | 1 | 1.971 | 1.546–3.548 |
| MeDeT-style | 5.75 | 1.051 | 1 | 2.040 | 1.566–4.594 |
| Scratch50 | 5.75 | 1.051 | 1 | 2.016 | 1.569–5.096 |
| Meta+NAS-lite | 50.09 | 1.225 | 12 | 14.816 | 6.525–37.954 |
| Zero-shot NAS | 78.81 | 0.157 | 0 | 2.667 | 0.977–4.480 |
| Zero-shot NAS + FT | 57.57 | 1.633 | 12 | 15.042 | 5.896–25.108 |
| H-Meta-NAS-based | 23.14 | 1.797 | 12 | 11.208 | 4.485–19.314 |
| MSA-DTI | 34.73 | 0.512 | 7 | 8.124 | 6.605–9.617 |

Resource counts are means over 120 medium-budget tasks. Adapt. counts candidate adaptation runs, each with 50 updates. Timing uses five repetitions for each of 12 horizon/support/budget settings on Cal VM 1204. Ranges span setting means.

### 5.5. Ablation Study

Table 4 examines candidate selection, the relative margin, reference designation, and source initialization. Applicable margins are calibrated separately using the same calibration pool, grid, and empirical harm criterion. The full method reduces MSE from 0.004669 for adapted A55 alone to 0.004488. Removing the margin increases MSE to 0.004537 and harmful selections from five to 15. A55 alone nevertheless has a slightly smaller task-level CVaR90 (0.036736) than the full method (0.036825).

Table 4: Component controls at the medium budget (errors in $10^{-2}$ units).

| Variant | $\tau$ | MSE | MAE | Worst-10% | CVaR90 | Harm |
|---|---|---|---|---|---|---|
| MSA-DTI | 0.05 | 0.4488 | 2.0543 | 2.8011 | 3.6825 | 4.17% |
| Reference only (A55) | — | 0.4669 | 2.3874 | 2.8904 | 3.6736 | 0.00% |
| Without relative margin | 0 | 0.4537 | 2.0376 | 2.8531 | 3.7230 | 12.50% |
| Fixed A57-PT reference | 0 | 0.4537 | 2.0376 | 2.8531 | 3.7230 | 12.50% |
| Reference only (A57-PT) | — | 0.4820 | 2.3692 | 2.9974 | 3.8389 | 0.00% |
| Two A57 initializations | 0.1 | 0.4749 | 2.2552 | 2.9509 | 3.8389 | 0.83% |
| Alternative source recipe | 0.2 | 0.4556 | 2.0905 | 2.8203 | 3.6736 | 0.00% |

Harm is relative to each row's own adapted reference over all 120 tasks. Each applicable margin is calibrated on Cal20. The fixed-A57 control changes reference designation and recalibrates its margin.

With A57-PT fixed as the reference and its own calibrated margin $\tau = 0$, MSE is 0.004537. The development-selected A55 policy reduces this value by 1.079%, with improvements for five VMs and ties for 25. This comparison measures the reference policy together with its calibrated margin. Selecting between the two A57 initializations yields MSE 0.004749; the broader bank reaches 0.004488. Replacing the alternative checkpoints with an alternative source-training recipe, while retaining A55, yields 0.004556 at $\tau = 0.2$.

Additional matched-optimizer controls use SGD, MSE, and 50 updates. In shared 12-candidate shortlists containing A55, calibrated selection yields MSE 0.004472, while direct validation minimization yields 0.004429. Direct validation minimization has the smaller MSE in this expanded bank, with zero harmful selections for both controls.

## 6. Conclusion

This paper presented MSA-DTI for digital twin model instantiation with few-shot target adaptation under operation-count and parameter-count limits. The method filters reusable source-trained candidates, compares them after common-budget adaptation, and applies a calibrated replacement rule around a development-selected reference. On the Bitbrains workload trace, MSA-DTI reduces aggregate MSE by 6.88% relative to PT+FT and satisfies both deployment limits in all evaluated tasks. The ablations show how candidate diversity, reference designation, and margin calibration affect prediction error and harmful selections. Source-training sensitivity also shows variation across pools and seeds. Candidate adaptation incurs additional target-side cost, and the present retrospective evidence is limited to one workload domain. Future work will examine independent workload domains and measure feasibility-pruned instantiation in deployed DT platforms.

## Funding

This work is supported by the Major Science and Technology Projects in Qinghai Province (2024-GX-A3).

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

The authors acknowledge Bitbrains IT Services Inc. for providing the workload traces and the Grid Workloads Archive for making them available [33, 34].

## Data Availability

The Bitbrains workload trace is available from the Grid Workloads Archive at https://atlarge-research.com/gwa-t-12/. The accompanying materials provide the task-role lists, feature normalization, candidate specifications, per-task results, plot-ready data, and figure-generation scripts. The selection-rule code, experimental-figure scripts, and manuscript sources are available in Release v1.3.3 at https://github.com/w924871681/paper-code-DT-Twin/releases/tag/v1.3.3.

## Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

OpenAI ChatGPT and Codex assisted with language editing, manuscript organization, and code for experimental processing and visualization. The authors are responsible for verifying the methods, results, figures, and references and for the content of the article.

## References

[1] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2230–2254, 4th Quart. 2022, doi: 10.1109/COMST.2022.3198273.

[2] S. Mihai et al., "Digital twins: A survey on enabling technologies, challenges, trends and future prospects," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2255–2291, 4th Quart. 2022, doi: 10.1109/COMST.2022.3208773.

[3] H. Xu, J. Wu, Q. Pan, X. Guan, and M. Guizani, "A survey on digital twin for Industrial Internet of Things: Applications, technologies and tools," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2569–2598, 4th Quart. 2023, doi: 10.1109/COMST.2023.3297395.

[4] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019, doi: 10.1109/TII.2018.2873186.

[5] J. Li et al., "Digital twin-enabled service provisioning in edge computing via continual learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7335–7350, 2024, doi: 10.1109/TMC.2023.3332668.

[6] H. Sartaj, S. Ali, and J. M. Gjøby, "MeDeT: Medical device digital twins creation with few-shot meta-learning," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 6, Art. no. 158, pp. 1–36, 2025, doi: 10.1145/3708534.

[7] A. Vettoruzzo, M.-R. Bouguelia, J. Vanschoren, T. Rögnvaldsson, and K. C. Santosh, "Advances and challenges in meta-learning: A technical review," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 7, pp. 4763–4779, 2024, doi: 10.1109/TPAMI.2024.3357847.

[8] H. Gharoun, F. Momenifar, F. Chen, and A. H. Gandomi, "Meta-learning approaches for few-shot learning: A survey of recent advances," *ACM Comput. Surv.*, vol. 56, no. 12, Art. no. 294, pp. 1–41, 2024, doi: 10.1145/3659943.

[9] M. Tan et al., "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 2820–2828, doi: 10.1109/CVPR.2019.00293.

[10] B. Wu et al., "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 10734–10742, doi: 10.1109/CVPR.2019.01099.

[11] Y. Zhao, L. Wang, and T. Guo, "Multi-objective neural architecture search by learning search space partitions," *J. Mach. Learn. Res.*, vol. 25, no. 177, pp. 1–41, 2024.

[12] Y. Zhao, X. Gao, I. Shumailov, N. Fusi, and R. Mullins, "Rapid model architecture adaption for meta-learning," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 18721–18732, doi: 10.52202/068431-1360.

[13] D. Lian et al., "Towards fast adaptation of neural architectures with meta learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[14] P. Eustratiadis, L. Dudziak, D. Li, and T. Hospedales, "Neural fine-tuning search for few-shot learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[15] B. Qin et al., "Machine and deep learning for digital twin networks: A survey," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34694–34716, Nov. 2024, doi: 10.1109/JIOT.2024.3416733.

[16] A. Hakiri, A. Gokhale, S. Ben Yahia, and N. Mellouli, "A comprehensive survey on digital twin for future networks and emerging Internet of Things industry," *Comput. Netw.*, vol. 244, Art. no. 110350, 2024, doi: 10.1016/j.comnet.2024.110350.

[17] Y. Wu, K. Zhang, and Y. Zhang, "Digital twin networks: A survey," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 13789–13804, Sep. 2021, doi: 10.1109/JIOT.2021.3079510.

[18] M. Emami Khansari and S. Sharifian, "A deep reinforcement learning approach towards distributed Function as a Service (FaaS) based edge application orchestration in cloud-edge continuum," *J. Netw. Comput. Appl.*, vol. 233, Art. no. 104042, 2025, doi: 10.1016/j.jnca.2024.104042.

[19] D. Hortelano et al., "A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems," *J. Netw. Comput. Appl.*, vol. 216, Art. no. 103669, 2023, doi: 10.1016/j.jnca.2023.103669.

[20] J. Son, S. Lee, and G. Kim, "When meta-learning meets online and continual learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 1, pp. 413–432, 2025, doi: 10.1109/TPAMI.2024.3463709.

[21] L. M. Zintgraf, K. Shiarli, V. Kurin, K. Hofmann, and S. Whiteson, "Fast context adaptation via meta-learning," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, vol. 97, 2019, pp. 7693–7702.

[22] Y. Zhao, T. Zhang, J. Li, and Y. Tian, "Dual adaptive representation alignment for cross-domain few-shot learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 11720–11732, 2023, doi: 10.1109/TPAMI.2023.3272697.

[23] J. Wu, X. Liu, X. Yin, T. Zhang, and Y. Zhang, "Task-adaptive prompted transformer for cross-domain few-shot learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 6, pp. 6012–6020, 2024, doi: 10.1609/aaai.v38i6.28416.

[24] W. Wang, L. Duan, Y. Wang, J. Fan, and Z. Zhang, "MMT: Cross domain few-shot learning via meta-memory transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 15018–15035, Dec. 2023, doi: 10.1109/TPAMI.2023.3306352.

[25] Z. Zuo, Y. Huang, Z. Li, Y. Jiang, and C. Liu, "Mixed contrastive transfer learning for few-shot workload prediction in the cloud," *Computing*, vol. 107, no. 1, Art. no. 5, 2025, doi: 10.1007/s00607-024-01366-y.

[26] K. You, Y. Liu, J. Wang, and M. Long, "LogME: Practical assessment of pre-trained models for transfer learning," in *Proc. 38th Int. Conf. Mach. Learn.*, vol. 139, 2021, pp. 12133–12143. https://proceedings.mlr.press/v139/you21b.html.

[27] K. You, Y. Liu, Z. Zhang, J. Wang, M. I. Jordan, and M. Long, "Ranking and tuning pre-trained models: A new paradigm for exploiting model hubs," *J. Mach. Learn. Res.*, vol. 23, no. 209, pp. 1–47, 2022. https://www.jmlr.org/papers/v23/21-1251.html.

[28] J. Bornschein, F. Visin, and S. Osindero, "Small data, big decisions: Model selection in the small-data regime," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, vol. 119, 2020, pp. 1035–1044.

[29] A. Lacoste, M. Marchand, F. Laviolette, and H. Larochelle, "Agnostic Bayesian learning of ensembles," in *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, vol. 32, 2014, pp. 611–619.

[30] Y. Zhao, L. Wang, Y. Tian, R. Fonseca, and T. Guo, "Few-shot neural architecture search," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 12707–12718.

[31] G. Li, D. Hoang, K. Bhardwaj, M. Lin, Z. Wang, and R. Marculescu, "Zero-shot neural architecture search: Challenges, solutions, and opportunities," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 7618–7635, Dec. 2024, doi: 10.1109/TPAMI.2024.3395423.

[32] M.-T. Wu, H.-I. Lin, and C.-W. Tsai, "A training-free neural architecture search algorithm based on search economics," *IEEE Trans. Evol. Comput.*, vol. 28, no. 2, pp. 445–459, Apr. 2024, doi: 10.1109/TEVC.2023.3264533.

[33] S. Shen, V. van Beek, and A. Iosup, "Statistical characterization of business-critical workloads hosted in cloud datacenters," in *Proc. 15th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGrid)*, 2015, pp. 465–474, doi: 10.1109/CCGrid.2015.60.

[34] @Large Research, "GWA-T-12 BitBrains," Grid Workloads Archive. [Online]. Available: https://atlarge-research.com/gwa-t-12/. Accessed: Oct. 27, 2026.